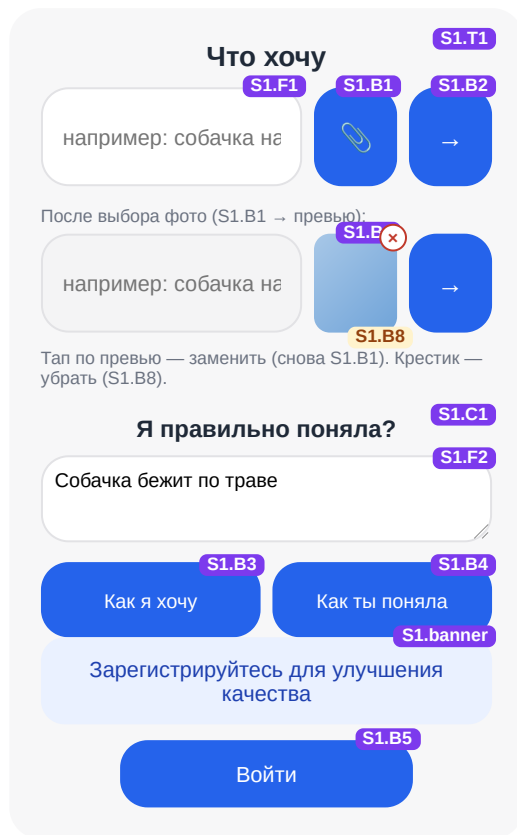


S1 — Главный экран (MainScreen.kt)

Макет и описание процедур для каждой кнопки



```
S1.B1.onclick() --> (нет перехода)
S1.B1.onclick() = MainScreen.kt_launchPhotoPicker()
/* Открывает системный Photo Picker, без runtime-разрешений.
   Тот же launcher — и для первого выбора (кнопка-скрепка), и
   для
   замены (тап по превью, когда фото уже выбрано).
   selectedPhotoUri — URI выбранного фото; пока != null,
   кнопка
   заменяется превью (thumbnailBitmap, сэмплированная копия).
*/
photoPickerLauncher.launch(PickVisualMediaRequest(ImageOnly))
selectedPhotoUri = <результат выбора>
```

```
S1.B8.onclick() --> S1.update()
S1.B8.onclick() = MainScreen.kt (крестик поверх превью)
/* Виден только когда фото уже выбрано. Полностью убирает
   фото,
   ничего не открывая заново — в отличие от тапа по самому
   превью
   (это S1.B1, замена). */
selectedPhotoUri = null
thumbnailBitmap = null
```

```
S1.B2.onclick() --> S1.C1.open()
S1.B2.onclick() = MainScreen.kt_onExpandPromptClick() [P1]
/* shortPrompt — текст из поля S1.F1.
   selectedPhotoUri → photoBytes, если фото приложено.
   onExpandPrompt — вызывает
   GeminiPromptExpansionService.kt_expandPrompt()
   (Gemini API, gemini-2.5-flash, бесплатный тариф).
   isExpanding — пока true, поверх экрана LoadingOverlay (01).
   expandedPrompt — из Success, показывает S1.C1.
   expansionError — из Failed, показывается под кнопками. */
isExpanding = true
result = onExpandPrompt(shortPrompt, photoBytes)
isExpanding = false
Success → expandedPrompt = result.expandedPrompt
Failed → expansionError = result.reason
```

```

S1.B3.onclick() --> S2.open()
    S1.B3.onclick() = AppNavigation.kt_onSubmit(prompt =
shortPrompt) [P4]
    /* prompt – исходный текст, без правок из S1.C1.
    imageGenerationService – по qualityLevel: QLEVEL_1
(оплачено) →
        GeminiProImageGenerationService.kt ("Nano Banana Pro",
платно),
        иначе GeminiFlashImageGenerationService.kt ("Nano Banana",
бесплатно).
    isLoading – пока true, на S2 показан LoadingOverlay (01).
*/

    lastPrompt = prompt; isLoading = true
    navController.navigate("result")
    imageGenerationService = (qualityLevel == QLEVEL_1) ?
proImageService : flashImageService
    result = imageGenerationService.generateImage(prompt)
    Success → resultImageBytes = hasWatermark ?
Watermark.kt_apply(...) : result.imageBytes
        database.profileHistoryDao().insert(...) // запись в
S5.C1
    Failed → contentErrorMessage = result.reason

```

```

S1.B4.onclick() --> S2.open()
    S1.B4.onclick() = AppNavigation.kt_onSubmit(prompt =
editableText) [P4]
    /* То же самое, что S1.B3, но prompt = editableText
(подтверждено/поправлено в S1.F2). */
    lastPrompt = prompt; isLoading = true
    navController.navigate("result")
    result = imageGenerationService.generateImage(prompt)
    Success → resultImageBytes = hasWatermark ?
Watermark.kt_apply(...) : result.imageBytes
        database.profileHistoryDao().insert(...) // запись в
S5.C1
    Failed → contentErrorMessage = result.reason

```

```

S1.B5.onclick() --> S4.open()
    S1.B5.onclick() = AppNavigation.kt_onLoginClick()
    /* Открыть экран входа. Параметров нет. */
    navController.navigate("login")

```

```

S1.B6.onclick() --> S1.update()
    S1.B6.onclick() = AppNavigation.kt_onLogoutClick()
    /* accessState (тип AccessState из PricingConfig.kt) –
определяет уровень качества и вид экрана. */
    accessState = AccessState.GUEST

```

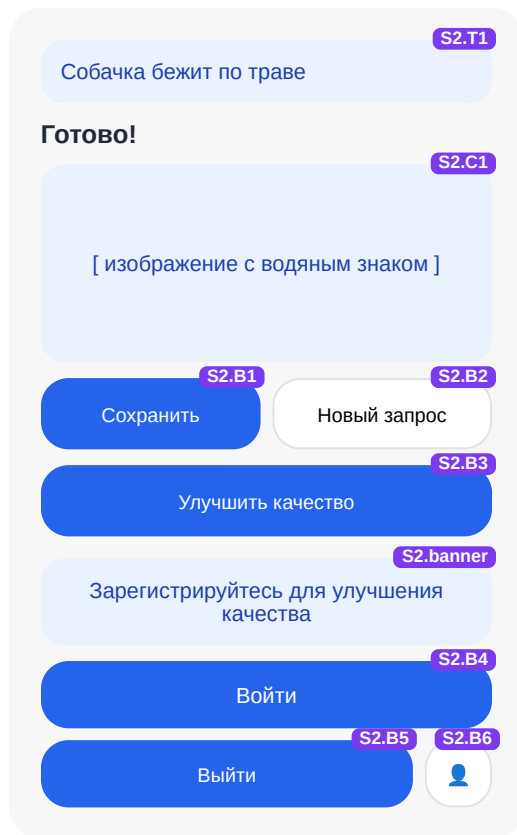
```

S1.B7.onclick() --> S5.open()
    S1.B7.onclick() = AppNavigation.kt_onProfileClick()
    /* Открывает экран профиля. Видна только зарегистрированным.
*/
    navController.navigate("profile")

```

S2 — Экран результата (ResultScreen.kt)

Макет и описание процедур для каждой кнопки



```
S2.B1.onclick() --> (нет перехода)
S2.B1.onclick() = AppNavigation.kt_saveResultImage(bytes)
/* MediaStore, через ImageSaver.kt. API 26-28 — сначала
runtime-разрешение WRITE_EXTERNAL_STORAGE, API 29+ —
сразу.
saveMessage/saveIsError — результат, показан под кнопками
S2. */
needsPermission ? запросить разрешение :
ImageSaver.kt_saveToGallery(...)
Success → saveMessage = "Сохранено в галерею"
Failed → saveMessage = result.reason, saveIsError = true
```

```
S2.B2.onclick() --> S1.open()
S2.B2.onclick() = AppNavigation.kt_onNewRequest()
/* Вернуться на главный экран. Параметров нет. */
navController.popBackStack()
```

```
S2.B3.onclick() --> S3.open()
S2.B3.onclick() = AppNavigation.kt_onUpgradeClick()
/* Видна только при hasWatermark == true (из accessState
через PricingConfig.kt_qualityLevelFor). Параметров нет.
*/
navController.navigate("paywall")
```

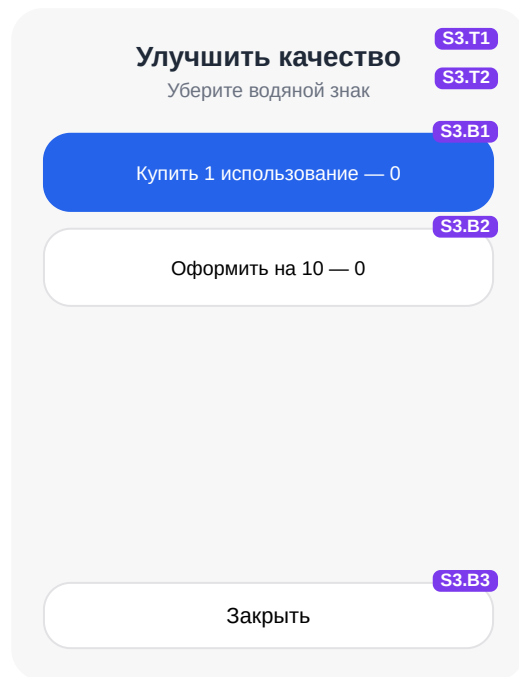
```
S2.B4.onclick() --> S4.open()
S2.B4.onclick() = AppNavigation.kt_onLoginClick()
/* Открыть экран входа. Параметров нет. */
navController.navigate("login")
```

```
S2.B5.onclick() --> S2.update()
S2.B5.onclick() = AppNavigation.kt_onLogoutClick()
/* То же самое, что S1.B6. */
accessState = AccessState.GUEST
```

```
S2.B6.onclick() --> S5.open()
S2.B6.onclick() = AppNavigation.kt_onProfileClick()
/* То же самое, что S1.B7 — открывает экран профиля. */
navController.navigate("profile")
```

S3 — Экран оплаты (PaywallScreen.kt)

Макет и описание процедур для каждой кнопки



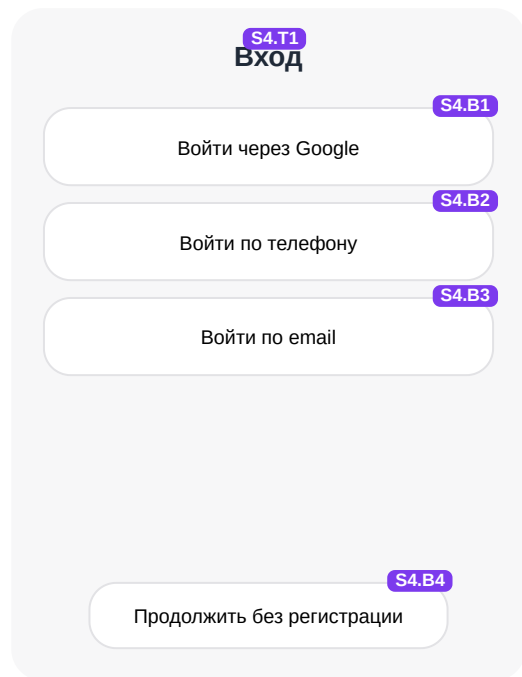
```
S3.B1.onclick() --> S2.open() | S3.update()
S3.B1.onclick() = AppNavigation.kt_onBuySingle()
/* activity – нужен провайдеру для запуска оплаты.
   SINGLE_USE – тип покупки (PurchaseType).
   isRegistered – какой товар: guest_* или user_*.
   result – ответ PaymentManager.kt_purchase() (P5), только
   после PurchaseVerificationService.kt_verify() (сейчас –
   заглушка). */
paymentManager.purchase(activity, SINGLE_USE, isRegistered)
Success → accessState = *_PAID_1,
database.profileHistoryDao().insert(...), возврат на S2
Failed → paymentErrorMessage = result.reason
```

```
S3.B2.onclick() --> S2.open() | S3.update()
S3.B2.onclick() = AppNavigation.kt_onBuySubscription()
/* Параметры как у S3.B1, но SUBSCRIPTION_10 (P6). */
paymentManager.purchase(activity, SUBSCRIPTION_10,
isRegistered)
Success → accessState = USER_PAID_10,
database.profileHistoryDao().insert(...), возврат на S2
Failed → paymentErrorMessage = result.reason
```

```
S3.B3.onclick() --> S2.open()
S3.B3.onclick() = AppNavigation.kt_onClose()
/* paymentErrorMessage – сбрасывается, чтобы не всплыла
   снова при следующем открытии S3. */
paymentErrorMessage = null
navController.popBackStack()
```

S4 — Экран входа (LoginScreen.kt)

Макет и описание процедур для каждой кнопки

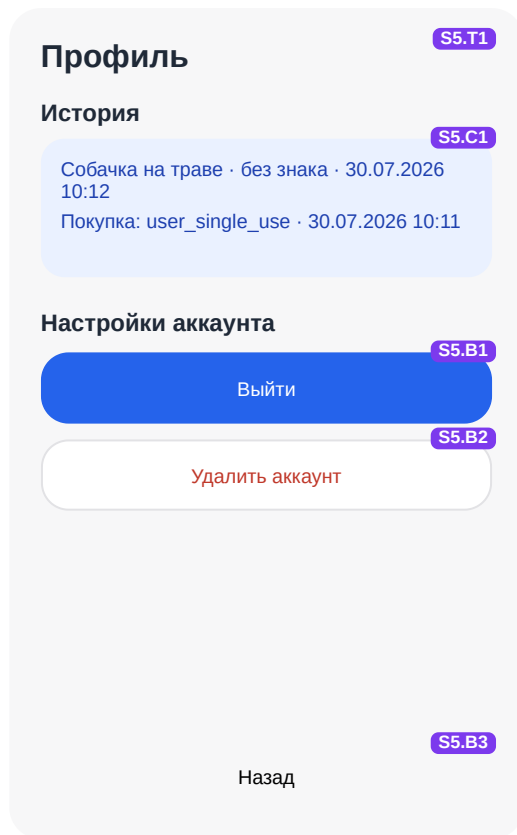


```
S4.B1/BE2/B3.onclick() --> (предыдущий экран).open()
S4.B1/BE2/B3.onclick() = AppNavigation.kt_onGoogleLogin() /
                           onPhoneLogin() / onEmailLogin()
/* Три способа сейчас ведут к одному результату (заглушка,
   без реальной проверки). accessState – переводится в USER
   (зарегистрирован, ещё не платил). */
accessState = AccessState.USER
navController.popBackStack()
```

```
S4.B4.onclick() --> (предыдущий экран).open()
S4.B4.onclick() = AppNavigation.kt_onContinueAsGuest()
/* Без изменения accessState – пользователь остаётся
   гостем. */
navController.popBackStack()
```

S5 — Экран профиля (ProfileScreen.kt)

Макет и описание процедур для каждой кнопки



```
S5.B1.onclick() --> S1.open()
S5.B1.onclick() = AppNavigation.kt_onLogout() [внутри
composable("profile")]
/* Сбрасывает accessState и возвращается на предыдущий
экран. */
accessState = AccessState.GUEST
navController.popBackStack()
```

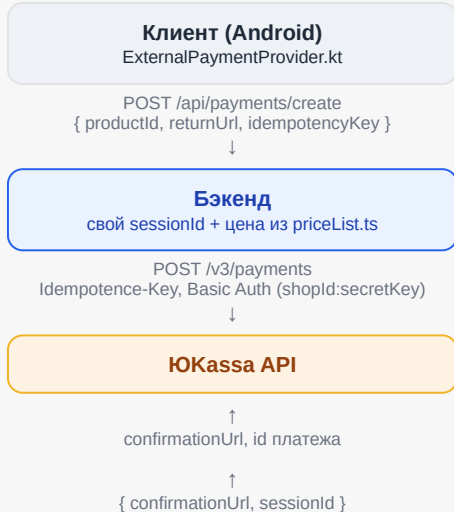
```
S5.B2.onclick() --> S1.open()
S5.B2.onclick() = AppNavigation.kt_onDeleteAccount() [P11,
заглушка]
/* Реального аккаунта/бэкенда для пользователей ещё нет —
"удаление" сбрасывает accessState и чистит персистентную
историю (Room, profile/db/). */
accessState = AccessState.GUEST
database.profileHistoryDao().clearAll()
navController.popBackStack()
```

```
S5.B3.onclick() --> (предыдущий экран).open()
S5.B3.onclick() = AppNavigation.kt_onBack()
/* Просто закрывает экран профиля, ничего не меняя. */
navController.popBackStack()
```

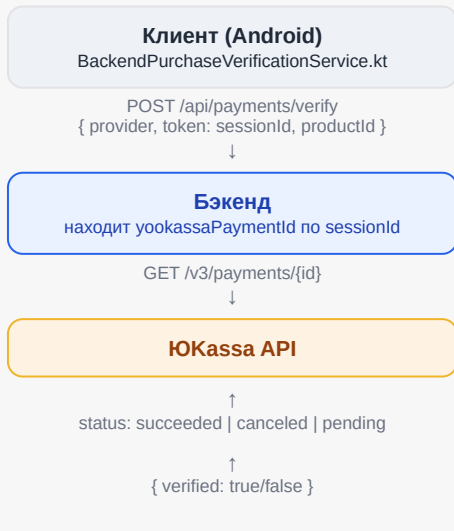
Бэкенд — оплата через ЮKassa (backend/src/server.ts)

Отдельный сервис (архив backend.zip), не часть Android-проекта. Полный контракт — docs/BACKEND-CONTRACT-YOOKASSA.md

Поток BE1 — создание платежа



Поток BE2 — проверка статуса



```
BE1: POST /api/payments/create
Вызывается из:
ExternalPaymentProvider.kt_createCheckoutSessionOnBackend() [клиент]
/* idempotencyKey — сгенерирован клиентом на этот запрос;
повторный
запрос с тем же ключом обязан вернуть ту же сессию, не создавая
второй платёж. */
server.ts_onPaymentsCreate(req)
/* price — только из priceList.ts по productId, сумма из
запроса
клиента игнорируется. sessionId генерируется ЗДЕСЬ, до вызова
ЮKassa — иначе нечем заполнить returnUrl ("курица и яйцо").
*/
existing = store.kt_findByIdempotencyKey(idempotencyKey)
existing != null → вернуть {existing.confirmationUrl,
existing.sessionId}
price = priceList.kt_getPriceForProduct(productId)
price == null → 400 "Неизвестный productId"
sessionId = crypto.randomUUID()
yookassaPayment = yookassa.kt_createYookassaPayment(price,
returnUrl+session, idempotencyKey)
store.kt_savePayment({sessionId, yookassaPaymentId:
yookassaPayment.id, ...})
Ответ → { confirmationUrl, sessionId }
```

```
BE2: POST /api/payments/verify
Вызывается из:
BackendPurchaseVerificationService.kt_verify() [клиент]
/* token — это НАШ sessionId из BE1, не id платежа ЮKassa
напрямую. */
server.ts_onPaymentsVerify(req)
record = store.kt_getBySessionId(token)
record == null → { verified: false }
record.productId != productId → { verified: false } //
подозрительно
yookassaPayment =
yookassa.kt_getYookassaPayment(record.yookassaPaymentId)
/* Единственный источник правды о статусе — этот запрос к
ЮKassa,
а НЕ то, что записано локально при создании (там всегда
"pending").
Возврат пользователя из браузера сам по себе ничего не значит
—
он мог просто закрыть вкладку, ничего не оплатив. */
status == "succeeded" → updateStatus(sessionId, "succeeded"); {
verified: true }
status == "canceled" → updateStatus(sessionId, "canceled"); {
verified: false }
status == "pending" → { verified: false } // ещё не завершено
```

Хранилище (store.ts) — JSON-файл, для старта/тестов.
Секреты ЮKassa (shopId/secretKey) — только в переменных окружения
бэкенда,
никогда в коде приложения. Подробности и деплой — backend/README.md.